AQUA: Activity- and Quantization-Aware Uniform Pruning for Spiking Neural Networks

***Abstract*—Spiking Neural Networks (SNNs) offer high energy efficiency through spatiotemporal sparsity and event-driven computation, but deploying them remains challenging due to irregular sparsity and workload imbalance. While careful pruning techniques can mitigate these challenges, existing methods ignore temporal activity patterns unique to SNNs or completely neglect the reintroduction of workload imbalance due to quantization. To address this, we propose AQUA, an Activity- and Quantization-Aware Uniform Pruning framework that aligns SNN sparsity with accelerator execution. AQUA introduces an activity-aware importance metric based on presynaptic firing statistics and postsynaptic membrane dynamics, and a quantization-aware rebalancing stage that preserves uniform workload after quantization by anticipating the quantization-induced sparsity drift. Across VGG-16 and ResNet-19 SNNs on static and neuromorphic datasets, AQUA achieves ∼92% sparsity with minimal accuracy loss while maintaining 97% PE utilization on VGG-16 and 93% on ResNet-19 after quantization, where prior SOTA methods fall to as low as 60%. Hardware simulation shows models sparsified by AQUA can lead to 60% energy consumption reduction compared to a dense baseline.**

I. Introduction

Spiking neural networks (SNNs) have been increasingly adopted in a broad range of applications ranging from image classification [1], and speech recognition [2] to natural language processing [3]. When combined with event-driven and highly parallel computation of neuromorphic processors, SNNs with sparse binary activations present a promising paradigm for energy-efficient intelligence. However, scaling SNNs to deep architectures and complex datasets still incurs substantial computation and memory costs, particularly when targeting resource-constrained accelerators that rely on aggressive sparsity and low-precision arithmetic for efficiency. This gap between the intrinsic sparsity potential of SNNs and the practical efficiency realized on hardware motivates the need for pruning and quantization schemes that are explicitly designed considering the parallel hardware execution model.

Pruning is a natural way to exploit spatial sparsity in SNNs, complementing their inherent temporal sparsity by eliminating redundant synaptic connections. Unstructured pruning methods can remove 80–95% of weights with modest accuracy loss [4] but generate highly irregular sparsity patterns that lead to severe workload imbalance across processing elements (PEs) in SIMD-style and neuromorphic accelerators. Structured pruning methods, in contrast, impose regular patterns (e.g., channel-wise or filter-wise) and have been shown to improve hardware utilization, but they often sacrifice accuracy or flexibility. Recent workload-balanced pruning schemes, such as μ-Ticket [5], move toward uniform workload distribution across PEs in floating-point space, yet they still overlook SNN-specific temporal dynamics and the impact of subsequent weight quantization.

Low-precision quantization is essential for deploying SNNs on practical accelerators, where INT4 or similar formats significantly reduce memory footprint and arithmetic cost. For instance, Intel Loihi [6] and Darwin3 [7] neuromorphic processors support flexible weight precisions from 1-9 bits, 1-16 bits respectively, while ODIN [8] only supports 3 bits. Quantization introduces an additional, often overlooked source of sparsity: weights near the quantizer zero point are mapped to zero, breaking previously balanced sparsity patterns and reducing PE utilization. In existing workload-balanced schemes, utilization can drop substantially after quantization as shown in Fig. 1, since quantization-induced zeros are not considered during pruning or rebalancing. For SNNs, this problem is exacerbated by the temporal variability of spike activity, which causes effective computation and energy consumption to be highly input- and neuron-dependent.

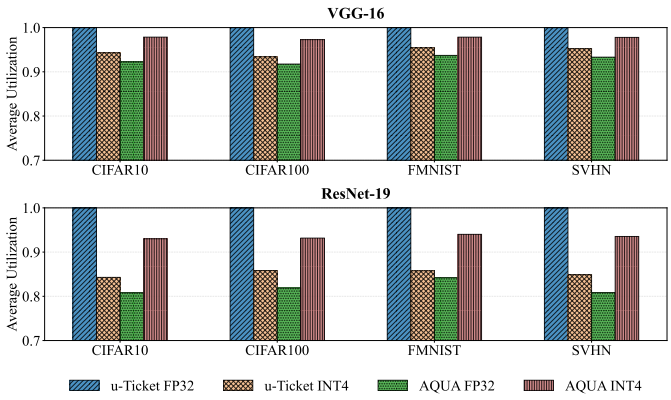

Fig. 1: Comparison between AQUA and μ-Ticket for workload-balance pruning in SNNs. AQUA maintains (or improves) utilization after quantization, whereas μ-Ticket experiences significant degradation.

These observations expose two key limitations of prior work when applied to SNNs: (1) pruning criteria that are agnostic to temporal spiking behavior treat all neurons as equally active and fail to capture which synapses truly dominate computation and accuracy, and (2) workload-balancing strategies that operate solely on floating-point weights cannot preserve uniform utilization after low-precision quantization. Consequently, there is a need for a pruning framework that is both activity-aware and quantization-aware, explicitly tar-

geting uniform workload in the quantized deployment setting while retaining high accuracy.

In this paper, we propose *Activity-Aware Uniform Pruning* (AQUA), a pruning framework for SNNs that is explicitly designed with parallel hardware and low-precision deployment in mind. AQUA enforces a uniform number of non-zero outgoing synapses per presynaptic neuron, introduces an activity-aware synaptic importance metric that leverages SNN spike statistics, and adds a quantization-aware rebalancing stage to maintain workload balance after low-precision mapping. Our main contributions are as follows:

- We introduce AQUA, a pruning framework for SNNs that jointly optimizes for sparsity, quantization, and hardware workload balance, enabling deep SNN deployment on resource-constrained accelerators with minimal accuracy loss.
- We develop novel activity-aware importance metrics and quantization-aware rebalancing techniques that incorporate SNN temporal dynamics such as firing rates and membrane potentials to preserve PE utilization after quantization.
- Through extensive evaluation on diverse benchmarks and a cycle-accurate simulator, we demonstrate AQUA's superior accuracy and hardware efficiency (e.g., $\geq$ 97% PE utilization) at high sparsity levels compared to prior SOTA methods like μ-Ticket.

II. Background and Related Work

A. Spiking Neuron Model

In this work, we adopt the widely used leaky integrate-and-fire (LIF) neuron model due to its simplicity and broad support in existing SNN accelerator designs. At each discrete timestep t, the membrane potential $u_j[t]$ of postsynaptic neuron j is updated as

$$u_j[t+1] = \beta \cdot u_j[t] + \sum_i w_{ij} s_i[t], \tag{1}$$

where $\beta \in (0, 1]$ is the leak factor, i indexes presynaptic neurons, w_{ij} is the synaptic weight from neuron i to j, and $s_i[t] \in \{0, 1\}$ denotes the binary spike of presynaptic neuron i at timestep t. After the leak-and-integrate update, the firing decision is given by

$$s_j[t+1] = \begin{cases} 1, & \text{if } u_j[t+1] > V_{th}, \\ 0, & \text{otherwise}, \end{cases} \tag{2}$$

where V_{th} is the firing threshold. After a spike, the membrane potential is reset; in this work we assume a hard reset to zero, which is commonly used in SNN training and simplifies hardware implementation.

B. SNN Sparsity and Weight Pruning

Spiking neural networks naturally exhibit two forms of sparsity: (1) temporal sparsity, where individual neurons fire at a low rate over the total simulation timesteps, and (2) spatial sparsity, where many synaptic connections can be removed with limited accuracy impact. Temporal sparsity arises from the binary, event-driven firing mechanism, while spatial sparsity is typically induced via pruning strategies adapted from ANNs or motivated by bio-plausible learning rules.

Fig. 2: Example illustrating how quantization disrupts the lane-level load balance achieved through careful pruning.

Unstructured pruning removes individual connections based on some importance criterion and has been widely explored for SNNs. Threshold-based schemes [5], [9], [10] prune weights whose magnitudes fall below fixed or learned thresholds, while bio-inspired methods [11] use local rules such as STDP to eliminate synapses with weak spike-time correlation between pre- and postsynaptic neurons. Activity-driven approaches [12] further identify and remove "silent" neurons that contribute little to network output. Prior works like [4] have demonstrated that we can achieve up to 97% sparsity without huge performance degradation on deep SNNs with more than 16 layers by adapting Lottery Ticket Hypothesis (LTH) to SNN. While these techniques can achieve very high sparsity ratios, they produce irregular, randomly distributed non-zero patterns that map poorly to SIMD-style accelerators and limit the achievable utilization and energy savings.

To improve hardware efficiency, structured pruning enforces regular sparsity patterns by removing groups of parameters such as filters, channels, or blocks. For example, [13] applies PCA to average membrane potentials to identify redundant filters on a per-layer basis, and recent methods such as μ-Ticket [5] explicitly target workload balance across processing elements (PEs) by iteratively pruning and regrowing weights under a balanced non-zero budget per hardware lane. Although these approaches address load imbalance more directly than unstructured methods, they typically operate in floating-point space and do not account for additional sparsity introduced by post-training weight quantization which can upset workload balance obtained from pruning as shown in Fig. 2.

In contrast to prior workload-balanced techniques based solely on weight magnitudes in floating-point space, our approach explicitly models SNN temporal activity and further includes a quantization-aware rebalancing stage to maintain PE utilization after low-precision mapping, as required by typical neuromorphic accelerators.

III. Activity- and Quantization-aware Uniform Pruning

A. Hardware-Aware Workload Balancing

Consider a weight matrix $W \in \mathbb{R}^{M \times N}$ mapped to a systolic array or SIMD architecture with P parallel Processing Elements (PEs). In a standard unstructured pruning regime,

Algorithm 1: Activity- and Quantization-aware Uniform Pruning (AQUA)

Input: SNN $f(x; \{\theta_l\}_{l=1}^{L})$, target sparsity s, pruning iterations K, train epochs E_{train}, importance weights (α, β), calibration batches B
Output: Pruned weights $\{\theta_l\}_{l=1}^{L}$ and binary masks $\{m_l\}_{l=1}^{L}$

```
// Phase 1: Dense pretraining
```
Initialize $m_l \leftarrow \mathbf{1}$ for all l ;
Train $f(x; \{\theta_l \odot m_l\}_{l=1}^{L})$ for E_{train} epochs ;
```
// Phase 2: Iterative Pruning and Recovery
```
for *iteration* $k = 1$ **to** K **do**
```
    // Step 2a: Collect activity statistics
```
    for $b = 1$ **to** B **do**
        Run forward pass to record presynaptic spikes and membrane potentials ;
```
    // Step 2b: Compute importance and apply global pruning
```
    for $l = 1$ **to** L **do**
        Compute importance $\mathcal{I}_l$ using (5) ;
    Calculate global threshold τ to prune $p\%$ of current weights ;
    Update masks: $m_l \leftarrow m_l \odot \mathbb{I}(\mathcal{I}_l > \tau)$ for all l ;
```
    // Step 2c: Quantization-aware Utilization Recovery
```
    for $l = 1$ **to** L **do**
        Compute quantization probability maps P_l based on bit-width ;
        Adjust m_l to balance PE workload using P_l (restoring uniformity) ;
```
    // Step 2d: Weight Rewinding and Retraining
```
    Rewind weights θ_l to early/initial state ;
    Train $f(x; \{\theta_l \odot m_l\}_{l=1}^{L})$ for E_{train} epochs ;
return $\{\theta_l\}_{l=1}^{L}, \{m_l\}_{l=1}^{L}$;

removing weights based solely on magnitude leads to irregular sparsity patterns. When columns (or groups of output channels) are distributed across PEs, this irregularity causes severe workload imbalance: some PEs may be assigned many non-zero weights while others sit idle, waiting for the busiest PE to finish.

To address this, prior works such as [10] enforce *strict uniform sparsity* (e.g., fixed k non-zeros per column). While this guarantees perfect load balancing, it severely restricts the network's ability to retain critical connections in highly active or sensitive areas. Instead of hard constraints, we propose a *utilization-driven recovery* strategy. We allow the pruning algorithm to first select weights globally based on importance (resulting in unstructured sparsity), and then explicitly rebalance the workload by reviving critical weights in under-utilized PEs and pruning least-important weights in over-utilized PEs. This ensures high hardware efficiency without imposing rigid per-neuron constraints.

B. Synaptic Weight Importance Metric

To guide the pruning and rebalancing process, we require a metric that identifies which connections are most critical for SNN performance. A natural baseline is magnitude-based pruning, where low-magnitude weights are removed as in prior uniform sparsity and μ-Ticket schemes. However, such schemes treat all neurons as equally active and ignore SNN-specific temporal dynamics: a large weight from a rarely firing neuron contributes no effective computation, whereas a moderate weight from a frequently spiking neuron can dominate both energy and latency. Similarly, postsynaptic neurons that seldom approach their firing threshold contribute little to the network's functional behavior and can tolerate more aggressive pruning.

Motivated by these observations, we introduce a three-factor importance metric that combines synaptic weight magnitude, presynaptic activity, and postsynaptic proximity to firing threshold. In addition to $|\mathbf{w}_{ij}|$, we estimate the average firing rate of the presynaptic neuron over T timesteps and B calibration batches:

$$a_j = \frac{1}{BT} \sum_{b=1}^{B} \sum_{t=1}^{T} s_j(b,t) \tag{3}$$

where $s_j(b,t) \in \{0,1\}$ indicates a spike from neuron j at timestep t in batch b.

We further define a postsynaptic proximity factor that captures how often each neuron operates near its firing threshold:

$$p_i = \tanh\left(\frac{\bar{V}_i}{V_{\text{th}}}\right), \tag{4}$$

where $\bar{V}_i$ is the mean membrane potential of neuron i over the same calibration period and V_{th} is the firing threshold. Neurons with $p_i \approx 1$ frequently hover near threshold, indicating that their incoming connections are critical for generating spikes and should be pruned more conservatively.

Using these statistics, we define the importance of synaptic connection (i,j) as

$$\mathcal{I}_{ij} = (1-\alpha) \cdot |\mathbf{w}_{ij}| + \alpha \left(|\mathbf{w}_{ij}| \cdot a_j \cdot [1 + \beta(p_i - 1)]\right), \tag{5}$$

where $\alpha \in [0,1]$ balances magnitude and activity contributions, and $\beta \in [0,1]$ controls the influence of postsynaptic proximity. When $\alpha = 0$, the metric reduces to pure magnitude pruning; when $\alpha = 1$, it becomes purely activity-driven. In practice, we find that combining all three factors yields better trade-offs between accuracy and sparsity than magnitude-only or activity-only baselines.

C. Iterative Training and Pruning Flow

Algorithm 1 summarizes the AQUA flow, which follows the Iterative Magnitude Pruning (IMP) and Lottery Ticket Hypothesis paradigm. We first train a dense SNN for E_{pretrain} epochs. In each pruning round, we collect activity statistics (a_j, p_i) using calibration batches, compute global importance scores $\mathcal{I}_{ij}$ and prune the bottom $p\%$ of weights globally. Then, we apply the quantization-aware rebalancing (described in III-D) to restore load balance across PEs. To recover accuracy, we rewind the surviving weights to their values from an early training epoch (Late Rewinding) and retrain for E_{retrain} epochs following the approach by [4] and [5]. This iterative process allows the network to adapt its topology gradually, maintaining high accuracy even at extreme sparsity levels.

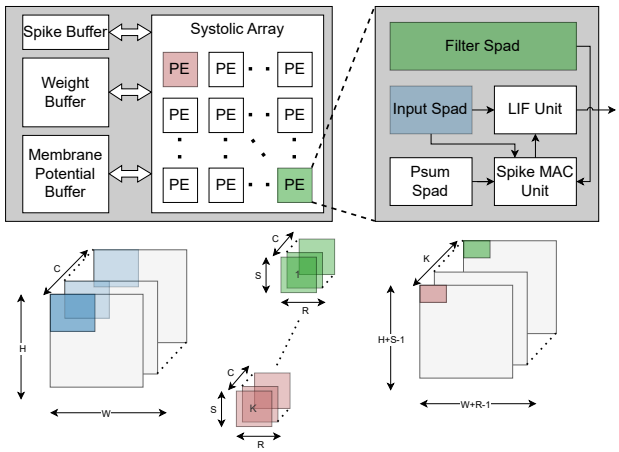

Fig. 3: Simplified view of SATA Architecture and its dataflow, where filters remain stationary inside PE.

D. Quantization-Aware Weight Rebalancing

Neuromorphic and accelerator platforms typically require SNN weights to be quantized to low precision (e.g., INT4) for deployment. However, quantization can reintroduce zero-valued weights after pruning, breaking the carefully enforced per-column non-zero counts and reducing PE utilization. To address this, we propose a quantization-aware rebalancing scheme that anticipates the effect of low-precision mapping and restores workload balance.

Concretely, we perform fake quantization to the target precision (e.g., INT4) during training and reason about each weight in quantized space rather than in floating-point space. Because surviving weights can drift toward the quantizer zero point during training, a hard threshold on $|w_{ij}|$ is brittle. Instead, we model the likelihood that a weight remains non-zero after quantization as

$$P\big(w_{ij} \neq 0\big) = \begin{cases} 1, & \text{if } |w_{ij}| \geq s_q, \\ \dfrac{|w_{ij}|}{s_q}, & \text{if } 0 < |w_{ij}| < s_q, \\ 0, & \text{if } |w_{ij}| = 0, \end{cases} \tag{6}$$

where s_q is the quantization step size for the corresponding layer or channel. Intuitively, weights whose normalized magnitude exceeds one almost certainly map to non-zero levels, very small weights are likely quantized to zero, and intermediate weights contribute proportionally in expectation.

During the recovery phase of each iteration, we calculate the expected workload for each PE chunk as the sum of these probabilities. If a PE's workload exceeds the layer average, we deactivate the active weights with the lowest $P(w \neq 0)$. Conversely, if a PE is under-utilized, we reactivate pruned weights that have the highest $P(w \neq 0)$. This ensures that the final sparse mask leads to balanced computation in the quantized hardware deployment.

E. Hardware Mapping and Utilization Metrics

We consider SATA [14], which is a reconfigurable systolic-based accelerator, as the hardware platform for inference. Fig. 3 illustrates the overview of SATA's architecture and dataflow. We map the pruned network weights to Processing Elements (PEs) by dividing each layer's weight matrix into PE chunks. Output channels are grouped and assigned to PEs, with each PE processing a subset of output channels. The workload W_{PE} for each PE chunk is computed as the sum of quantization-aware probability values for active weights. To measure how uniformly computation is distributed across PEs, we first compute the average workload $\bar{W} = \frac{1}{n_{PE}} \sum_{j=1}^{n_{PE}} W_j$ and maximum workload $W_{\max} = \max(W_j)$ across all PEs. The utilization metric is defined as :

$$U = 1 - \frac{W_{\max} - \bar{W}}{W_{\max}} \cdot \frac{n_{PE}}{n_{PE} - 1}$$

Utilization ranges from 0 to 1, where $U \approx 1$ indicates balanced workloads (all PEs have similar load) and $U < 1$ indicates imbalance (some PEs are overloaded while others are underutilized). The average layerwise utilization is computed as the mean utilization across all output channels or PE groups within a layer, providing a single metric per layer. Low utilization leads to wasted cycles and increased latency, as the slowest PE determines overall execution time.

IV. Evaluation

A. Methodology

We evaluate AQUA with VGG-16 [15] and ResNet-19 [16] deep SNNs on both static datasets—CIFAR-10 [17], CIFAR-100 [17], FMNIST [18], and SVHN [19]—as well as neuromorphic datasets—DVS-CIFAR10 [20] and DVS-Gesture [1]. We implement the networks using PyTorch [21] and SpikingJelly [22]. Unless otherwise specified, we train the models with the settings in Table I. We mainly compare against μ-Ticket [5], which is the previous state of the art on uniform pruning for SNNs.

For hardware efficiency evaluation, we use an SNN architecture simulator, SATA-Sim [14], which internally uses ScaleSim [23] for obtaining cycle statistics and CACTI 7.0 [24] for memory modeling. We keep the original architecture of SATA, synthesized at 400 MHz using 65 nm CMOS technology, with 128 PEs, 144 KB weight buffer, 256 KB membrane potential buffer, and 32 KB spike buffer.

B. Importance Metric Hyperparameters

To study the effect of hyperparameters α and β used in importance metric computation for pruning on model accuracy, we sweep the accuracy values for VGG-16 on CIFAR-10 for different hyperparameter configurations. Fig. 4 reveals a non-linear interaction between α and β, with accuracy ranging from 85.59% to 86.59% across the hyperparameter space. The optimal configuration occurs at $\alpha = 0.75$ and $\beta = 0.50$, achieving 86.59% accuracy and representing a 0.64 percentage point improvement over the magnitude-only baseline ($\alpha = 0$). Notably, the same $\alpha = 0.75$ value produces

TABLE I: SNN Training Hyperparameters

Parameter	Value
Timestep	4
Neuron model	LIF
Membrane potential leak	0.75
Firing threshold	1
Reset mode	hard
Training batch size	128
Optimizer	SGD
Learning rate	1E–1
Weight decay	5E–4
Momentum	0.9
Total training epochs	150
Pruning rounds	10
Sampling batch size	100

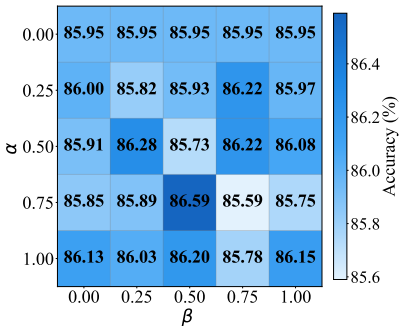

Fig. 4: Analysis on the sensitivity of α and β hyperparameters on accuracy of VGG-16 for CIFAR-10.

both the best (86.59% at $\beta = 0.50$) and worst (85.59% at $\beta = 0.75$) results, indicating that β selection becomes critical when activity-aware pruning is emphasized, and suggesting that moderate values of both hyperparameters ($\alpha \in [0.5, 0.75]$, $\beta \in [0.25, 0.5]$) provide the most robust performance.

C. Neuromorphic Dataset Evaluation

We further evaluate AQUA on two neuromorphic datasets: DVS-CIFAR10, and DVS-Gesture with the $\alpha = 0.75$ and $\beta = 0.5$ combination, which achieved the highest accuracy results for VGG-16 on CIFAR-10 in Table II. While u-Ticket outperforms AQUA marginally by $+1.6\%$ for DVS-CIFAR10, AQUA achieves significantly higher accuracy ($+5.21\%$) for DVS-Gesture dataset. In terms of INT4 utilization, AQUA consistently outperforms u-Ticket with around 2% gap. Note that AQUA can also achieve 100% utilization for deployment situations that do not require quantization, by simply falling back to balancing in the floating-point space.

TABLE II: Evaluation on Neuromorphic Datasets with VGG-16

Dataset	Method	FP32 Acc.(%)	FP32 Avg. Util.	INT4 Avg. Util.
DVS-CIFAR10	μ-Ticket	**66.00**	1	95.98
	AQUA **(ours)**	64.40	94.16	**97.61**
DVS-Gesture	μ-Ticket	74.65	1	96.37
	AQUA **(ours)**	**79.86**	94.21	**98.51**

TABLE III: Accuracy, Sparsity, and Average Layer-wise Utilization: μ-Ticket vs. AQUA

Dataset	Method	Accuracy (%)		Sparsity (%)	
		FP32	INT4	FP32	INT4
VGG-16					
CIFAR-10	μ-Ticket	**84.96**	83.72	92.43	**92.49**
	AQUA **(ours)**	84.85	**83.89**	**92.45**	92.48
CIFAR-100	μ-Ticket	54.94	**52.79**	92.45	**92.61**
	AQUA **(ours)**	**55.37**	49.74	**92.47**	92.59
FMNIST	μ-Ticket	**92.83**	**92.42**	92.42	92.48
	AQUA **(ours)**	92.53	**86.05**	**92.46**	**92.49**
SVHN	μ-Ticket	**94.54**	**93.87**	92.42	92.48
	AQUA **(ours)**	94.41	93.65	**92.45**	**92.49**
ResNet-19					
CIFAR-10	μ-Ticket	**86.26**	**84.79**	**92.42**	**92.61**
	AQUA **(ours)**	86.13	84.78	92.32	92.44
CIFAR-100	μ-Ticket	55.65	**48.93**	**92.46**	**92.77**
	AQUA **(ours)**	**55.70**	**54.23**	92.32	92.58
FMNIST	μ-Ticket	**93.72**	88.99	**92.41**	**92.54**
	AQUA **(ours)**	92.98	**92.80**	92.33	92.41
SVHN	μ-Ticket	**95.71**	**95.35**	**92.42**	**92.60**
	AQUA **(ours)**	95.63	95.33	92.32	92.45

D. Energy Consumption and Hardware Utilization

Figure 5 compares the energy consumption between baseline dense model and AQUA for VGG-16 on CIFAR-10 at 92% sparsity. The high sparsity results in an $\sim$ 60% reduction on total energy consumption. The reduction is more pronounced in memory components as compressed weights lead to better DRAM bandwidth efficiency which have orders of magnitude higher energy consumption than SRAM buffers and compute units. For dense baseline, DRAM energy dominates the overall consumption,accounting for 48.5% of energy consumption. For AQUA, as more filter weights can be stored on-chip and reused, we see scratchpad (SPAD) and global weight buffer (GLB-weight) become the two biggest contributors responsible for 64.5% of the consumption when combined.

E. Comparison with Previous Work

To evaluate the effectiveness of AQUA in maintaining balanced workload under quantization, we compare PE utilization against μ-Ticket on VGG-16 and ResNet-19 across CIFAR-10, CIFAR-100, FMNIST, and SVHN, as shown in Fig. 6 and Table III. Training parameters are summarized in Table I. Note that while hardware utilization directly determines runtime and energy, sparsity only indirectly influences efficiency; thus, accuracy and utilization are the primary metrics, with FP32/INT4 sparsity reported for completeness.

The results show that although μ-Ticket achieves perfectly balanced utilization (100%) in FP32, its utilization degrades sharply after INT4 quantization: down to 84$\sim$95% for VGG-16, as quantization-induced zeros disrupt the uniform nonzero distribution. The degradation is more pronounced for the deeper ResNet-19, where utilization drops to 60%, effectively undoing much of the intended workload balance.

In contrast, AQUA maintains $> 97\%$ average layer-wise utilization for VGG-16 after quantization, demonstrating that quantization-aware rebalancing effectively compensates for

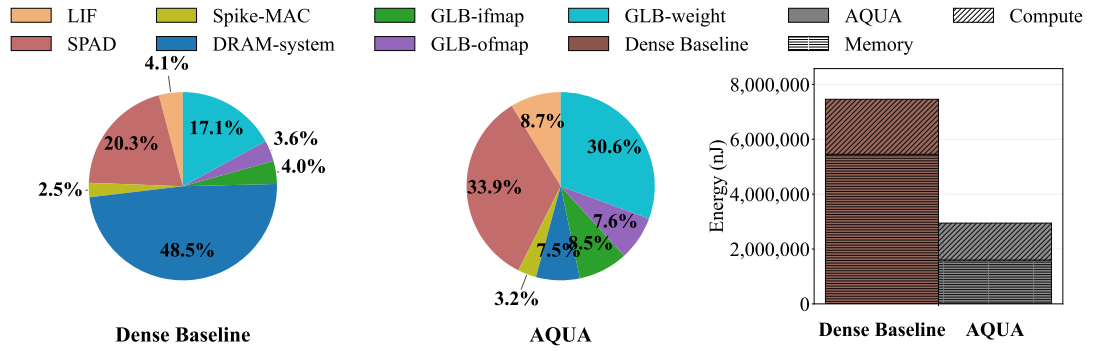

Fig. 5: Energy comparison for the dense baseline and AQUA for VGG-16 with 92% sparsity on CIFAR-10.

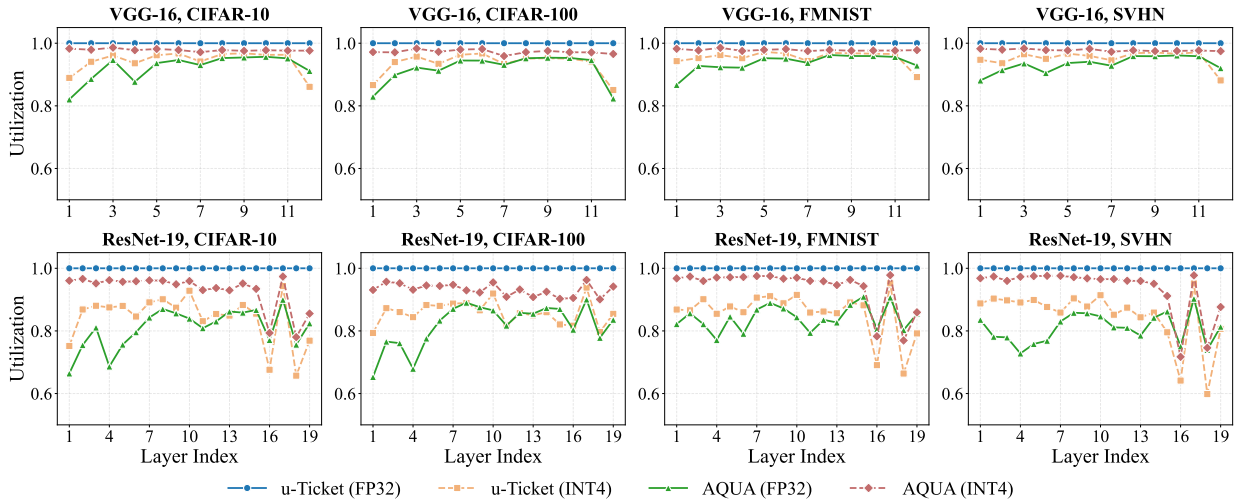

Fig. 6: Layer-wise PE utilization comparison between μ-Ticket and AQUA for VGG-16 and ResNet-19 architectures across multiple datasets in FP32 and INT4. For the primary metric (INT4 utilization), AQUA consistently outperforms the prior state of the art.

newly introduced zeros. For ResNet-19, where imbalance is amplified across depth, AQUA still sustains ∼93% utilization in INT4, indicating that its activity-aware importance and probabilistic workload modeling remain robust across architectures and datasets.

As shown in Table III, inference accuracy between the two methods is nearly identical. The largest gap appears between quantized models, where u-Ticket outperforms AQUA by 3.05% on CIFAR-100 and 6.37% on FMNIST for VGG-16 while AQUA outperforms u-Ticket by 5.3% and 3.81% respectively on same datasets for ResNet-19. Elsewhere, differences stay within ∼0.5% with no consistent winner. Sparsity levels are also similar (92∼93%), explaining the comparable accuracies but highlighting a key distinction: despite similar sparsity, AQUA aligns nonzero computations far more effectively, yielding substantially higher utilization.From a Pareto-optimality perspective, AQUA shifts the design point toward the desirable regime where high sparsity and high utilization coexist even after quantization. Note that we apply simple per-channel symmetric uniform quantizer with max-abs scaling and that further accuracy recovery from quantization error is possible with more elaborate techniques, which are orthogonal to our work.

V. Conclusion

This paper introduces AQUA, an activity- and quantization-aware uniform pruning framework that aligns SNN sparsity with the execution model of neuromorphic-style accelerators, rather than optimizing sparsity in isolation. By factoring in the input spike activity and membrane potential state of neurons during pruning, AQUA can achieve better accuracy recovery than magnitude-based pruning. AQUA also anticipates the additional sparsity induced by quantization during the training process to maintain the load-balance achieved from pruning. Experimental results on VGG-16 and ResNet-19 networks on various static and neuromorphic dataset demonstrate that AQUA can achieve on average 97% and 93% PE utilization after quantization, while previous SOTA method falls to as low as 60%. Model pruned by AQUA can reduce total energy consumption by up to 60% compared to dense baseline.

Acknowledgment

The authors thank the reviewers for their constructive comments.

REFERENCES

[1] A. Amir, B. Taba, D. Berg, T. Melano, J. McKinstry, C. Di Nolfo, T. Nayak, A. Andreopoulos, G. Garreau, M. Mendoza, J. Kusnitz, M. Debole, S. Esser, T. Delbruck, M. Flickner, and D. Modha, "A low power, fully event-based gesture recognition system," in *2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 7388–7397.

[2] J. Wu, E. Yılmaz, M. Zhang, H. Li, and K. C. Tan, "Deep spiking neural networks for large vocabulary automatic speech recognition," *Frontiers in Neuroscience*, vol. Volume 14 - 2020, 2020. [Online]. Available: https://www.frontiersin.org/journals/neuroscience/articles/10.3389/fnins.2020.00199

[3] Y. Li, Y. Lei, and X. Yang, "Spikeformer: A novel architecture for training high-performance low-latency spiking neural network," 2022. [Online]. Available: https://arxiv.org/abs/2211.10686

[4] Y. Kim, Y. Li, H. Park, Y. Venkatesha, R. Yin, and P. Panda, "Lottery ticket hypothesis for spiking neural networks," *arXiv preprint arXiv:2207.01382*, 2022.

[5] R. Yin, Y. Kim, Y. Li, A. Moitra, N. V. Satpute, A. Hambitzer, and P. Panda, "Workload-balanced pruning for sparse spiking neural networks," *IEEE Transactions on Emerging Topics in Computational Intelligence*, vol. 8, pp. 2897–2907, 2023. [Online]. Available: https://api.semanticscholar.org/CorpusID:256846987

[6] G. Orchard, E. P. Frady, D. B. D. Rubin, S. Sanborn, S. B. Shrestha, F. T. Sommer, and M. Davies, "Efficient neuromorphic signal processing with loihi 2," in *2021 IEEE Workshop on Signal Processing Systems (SiPS)*, 2021, pp. 254–259.

[7] D. Ma, X. Jin, S. Sun, Y. Li, X. Wu, Y. Hu, F. Yang, H. Tang, X. Zhu, P. Lin, and G. Pan, "Darwin3: a large-scale neuromorphic chip with a novel isa and on-chip learning," *National Science Review*, vol. 11, no. 5, p. nwae102, 03 2024. [Online]. Available: https://doi.org/10.1093/nsr/nwae102

[8] C. Frenkel, M. Lefebvre, J.-D. Legat, and D. Bol, "A 0.086-mm^2 12.7-pj/sop 64k-synapse 256-neuron online-learning digital spiking neuromorphic processor in 28-nm cmos," *IEEE Transactions on Biomedical Circuits and Systems*, vol. 13, no. 1, pp. 145–158, 2019.

[9] E. O. Neftci, B. U. Pedroni, S. Joshi, M. Al-Shedivat, and G. Cauwenberghs, "Stochastic synapses enable efficient brain-inspired learning machines," *Frontiers in Neuroscience*, vol. Volume 10 - 2016, 2016.

[10] S. Muralidharan, "Uniform sparsity in deep neural networks," in *Proceedings of Machine Learning and Systems*, D. Song, M. Carbin, and T. Chen, Eds., vol. 5. Curan, 2023, pp. 743–753. [Online]. Available: https://proceedings.mlsys.org/paper_files/paper/2023/file/f54c95da33c8572340f2f4b09b784e78-Paper-mlsys2023.pdf

[11] N. Rathi, P. Panda, and K. Roy, "Stdp-based pruning of connections and weight quantization in spiking neural networks for energy-efficient recognition," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 38, no. 4, pp. 668–677, 2019.

[12] R. Chen, H. Ma, S. Xie, P. Guo, P. Li, and D. Wang, "Fast and efficient deep sparse multi-strength spiking neural networks with dynamic pruning," in *2018 International Joint Conference on Neural Networks (IJCNN)*, 2018, pp. 1–8.

[13] S. S. Chowdhury, I. Garg, and K. Roy, "Spatio-temporal pruning and quantization for low-latency spiking neural networks," in *2021 International Joint Conference on Neural Networks (IJCNN)*, 2021, pp. 1–9.

[14] R. Yin, A. Moitra, A. Bhattacharjee, Y. Kim, and P. Panda, "Sata: Sparsity-aware training accelerator for spiking neural networks," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2022.

[15] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," *arXiv 1409.1556*, 09 2014.

[16] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," 06 2016, pp. 770–778.

[17] A. Krizhevsky, "Learning multiple layers of features from tiny images," *University of Toronto*, 05 2012.

[18] H. Xiao, K. Rasul, and R. Vollgraf, "Fashion-mnist: a novel image dataset for benchmarking machine learning algorithms," 2017. [Online]. Available: https://arxiv.org/abs/1708.07747

[19] Y. Netzer, T. Wang, A. Coates, A. Bissacco, B. Wu, and A. Ng, "Reading digits in natural images with unsupervised feature learning," *NIPS*, 01 2011.

[20] H. Li, H. Liu, X. Ji, G. Li, and L. Shi, "Cifar10-dvs: An event-stream dataset for object classification," *Frontiers in Neuroscience*, vol. Volume 11 - 2017, 2017. [Online]. Available: https://www.frontiersin.org/journals/neuroscience/articles/10.3389/fnins.2017.00309

[21] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Köpf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, *PyTorch: an imperative style, high-performance deep learning library*. Red Hook, NY, USA: Curran Associates Inc., 2019.

[22] W. Fang, Y. Chen, J. Ding, Z. Yu, T. Masquelier, D. Chen, L. Huang, H. Zhou, G. Li, and Y. Tian, "Spikingjelly: An open-source machine learning infrastructure platform for spike-based intelligence," *Science Advances*, vol. 9, no. 40, p. eadi1480, 2023. [Online]. Available: https://www.science.org/doi/abs/10.1126/sciadv.adi1480

[23] A. Samajdar, J. M. Joseph, Y. Zhu, P. Whatmough, M. Mattina, and T. Krishna, "A systematic methodology for characterizing scalability of dnn accelerators using scale-sim," in *2020 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. IEEE, 2020, pp. 58–68.

[24] R. Balasubramonian, A. B. Kahng, N. Muralimanohar, A. Shafiee, and V. Srinivas, "Cacti 7: New tools for interconnect exploration in innovative off-chip memories," *ACM Trans. Archit. Code Optim.*, vol. 14, no. 2, Jun. 2017. [Online]. Available: https://doi.org/10.1145/3085572